```
import pandas as pd

# Load the dataset
df = pd.read_csv('/content/Messy_data.csv')

# Display the shape and the first few rows
print(f"Dataset shape: {df.shape}")
display(df.head())
```

```
Dataset shape: (185, 9)
```

	OrderID	CustomerName	City	Product	Quantity	UnitPrice	OrderDate	OrderStatus
0	10001	Nikhil Jain	Mumbai	Laptop	3.0	50790.72	2026-01-18	Completed
1	10002	Simran Kaur	delhi	Monitor	1.0	10307.27	2026-01-28	Pending
2	10003	Zoya Ali	Delhi	Headphones	5.0	2439.64	2026-02-27	Delivered
3	10004	Priya Bansal	Delhi	Tablet	4.0	20945.65	02/03/2026	Pending
4	10005	NaN	Mumbai	Phone	4.0	15822.27	2026-02-14	Delivered

```
import os
print(os.listdir('/content'))
```

```
['.config', 'Messy_data.csv', 'sample_data']
```

```
import pandas as pd

# Correctly load the dataset and display the head
df = pd.read_csv('/content/Messy_data.csv')
df.head()
```

	OrderID	CustomerName	City	Product	Quantity	UnitPrice	OrderDate	OrderStatus
0	10001	Nikhil Jain	Mumbai	Laptop	3.0	50790.72	2026-01-18	Completed
1	10002	Simran Kaur	delhi	Monitor	1.0	10307.27	2026-01-28	Pending
2	10003	Zoya Ali	Delhi	Headphones	5.0	2439.64	2026-02-27	Delivered
3	10004	Priya Bansal	Delhi	Tablet	4.0	20945.65	02/03/2026	Pending
4	10005	NaN	Mumbai	Phone	4.0	15822.27	2026-02-14	Delivered

```
print("Dataset Shape:")
print(df.shape)

print("\nDataset Information:")
df.info()

print("\nMissing Values:")
print(df.isnull().sum())
```

```
Dataset Shape:
(185, 9)
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 185 entries, 0 to 184
Data columns (total 9 columns):
 #   Column        Non-Null Count  Dtype 
---  ------        --------------  ----- 
 0   OrderID       185 non-null    int64 
 1   CustomerName  183 non-null    object
 2   City          183 non-null    object
 3   Product       185 non-null    object
 4   Quantity      183 non-null    object
 5   UnitPrice     183 non-null    object
 6   OrderDate     183 non-null    object
 7   OrderStatus   183 non-null    object
 8   CustomerAge   183 non-null    object
dtypes: int64(1), object(8)
memory usage: 13.1+ KB

Missing Values:
OrderID         0
CustomerName    2
City            2
Product         0
Quantity        2
UnitPrice       2
OrderDate       2
OrderStatus     2
CustomerAge     2
dtype: int64
```

```
# Let's begin cleaning the dataset
import pandas as pd
import numpy as np

# Create a working copy
cleaned_df = df.copy()

# 1. Standardize text representation
cleaned_df['City'] = cleaned_df['City'].str.strip().str.title()
cleaned_df['OrderStatus'] = cleaned_df['OrderStatus'].str.strip().str.capitalize()

# 2. Clean numeric columns (removing non-digit characters if any)
for col in ['Quantity', 'UnitPrice', 'CustomerAge']:
    cleaned_df[col] = pd.to_numeric(cleaned_df[col].astype(str).str.replace(r'[^0-9.]',

# 3. Clean and convert OrderDate to standard datetime format
cleaned_df['OrderDate'] = pd.to_datetime(cleaned_df['OrderDate'], errors='coerce')

# Check the updated types and missing values
print("Cleaned Dataset Info:")
cleaned_df.info()
print("\nMissing values after cleaning:")
print(cleaned_df.isnull().sum())
display(cleaned_df.head())
```

```
Cleaned Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 185 entries, 0 to 184
Data columns (total 9 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   OrderID       185 non-null    int64
 1   CustomerName  183 non-null    object
 2   City          183 non-null    object
 3   Product       185 non-null    object
 4   Quantity      182 non-null    float64
 5   UnitPrice     182 non-null    float64
 6   OrderDate     179 non-null    datetime64[ns]
 7   OrderStatus   183 non-null    object
 8   CustomerAge   182 non-null    float64
dtypes: datetime64[ns](1), float64(3), int64(1), object(4)
memory usage: 13.1+ KB

Missing values after cleaning:
OrderID         0
CustomerName    2
City            2
Product         0
Quantity        3
UnitPrice       3
OrderDate       6
OrderStatus     2
CustomerAge     3
dtype: int64
```

	OrderID	CustomerName	City	Product	Quantity	UnitPrice	OrderDate	OrderStatus
0	10001	Nikhil Jain	Mumbai	Laptop	3.0	50790.72	2026-01-18	Completed
1	10002	Simran Kaur	Delhi	Monitor	1.0	10307.27	2026-01-28	Pending
2	10003	Zoya Ali	Delhi	Headphones	5.0	2439.64	2026-02-27	Delivered
3	10004	Priya Bansal	Delhi	Tablet	4.0	20945.65	NaT	Pending
4	10005	NaN	Mumbai	Phone	4.0	15822.27	2026-02-14	Delivered

```
print("Number of duplicate rows:")
print(df.duplicated().sum())
```

```
Number of duplicate rows:
5
```

```
# 4. Remove duplicate rows from our cleaned dataset
before_dedup = len(cleaned_df)
cleaned_df = cleaned_df.drop_duplicates()
after_dedup = len(cleaned_df)
print(f"Removed {before_dedup - after_dedup} duplicate rows. Remaining: {after_dedup}")

# 5. Handle missing values (e.g., dropping rows where key info is completely missing)
# We'll drop rows where crucial columns like 'CustomerName' or 'OrderDate' are null
cleaned_df = cleaned_df.dropna(subset=['CustomerName', 'OrderDate'])

print(f"Final clean dataset size: {len(cleaned_df)}")
display(cleaned_df.head(10))
```

```
Removed 5 duplicate rows. Remaining: 180
Final clean dataset size: 173
```

	OrderID	CustomerName	City	Product	Quantity	UnitPrice	OrderDate	OrderStatu
0	10001	Nikhil Jain	Mumbai	Laptop	3.0	50790.72	2026-01-18	Complete
1	10002	Simran Kaur	Delhi	Monitor	1.0	10307.27	2026-01-28	Pendir
2	10003	Zoya Ali	Delhi	Headphones	5.0	2439.64	2026-02-27	Delivere
5	10006	Priya Bansal	Delhi	Smartwatch	5.0	5768.41	2026-02-18	Complete
6	10007	Meera Joshi	Chennai	Headphones	1.0	2159.37	2026-01-30	Shippe
7	10008	Sara Thomas	Bengaluru	Phone	NaN	16801.06	2026-03-23	Shippe
8	10009	Tara Kapoor	Mumbai	Headphones	3.0	2651.37	2026-03-29	Complete
9	10010	Nikhil Jain	Pune	Headphones	2.0	2471.70	2026-02-04	Delivere
10	10011	Pooja Singh	Chennai	Laptop	2.0	60309.74	2026-02-10	Cancelle
11	10012	Kabir Singh	Bengaluru	Keyboard	NaN	1569.95	2026-02-20	Cancelle

```
columns = ['Quantity', 'UnitPrice', 'CustomerAge', 'City', 'OrderStatus']

for col in columns:
    print(f"\nUnique values in '{col}':")
    print(df[col].unique())
```

```
Unique values in 'Quantity':
['3.0' '1.0' '5.0' '4.0' 'three' '2.0' nan '2 pcs' '-3.0' '500.0']

Unique values in 'UnitPrice':
['50790.72' '10307.27' '2439.64' '20945.65' '15822.27' '5768.41' '2159.37'
 '16801.06' '2651.37' '2471.7' '60309.74' '1569.95' '2529.23' '2507.14'
 '10428.7' '18,500' '17647.73' nan '23732.57' '5743.14' '19252.3' '5847.0'
 '999999.0' '16513.37' '2178.24' '1506.37' '7055.01' '1460.0' '16334.59'
 '739.95' '60544.46' '7211.07' '748.46' '20658.27' '1297.56' '15670.02'
 '48102.73' '840.73' '755.88' '16451.03' '2338.74' '905.34' '20559.59'
 '7290.41' '52253.68' '10234.81' '19541.14' '1581.93' '24406.4' '11160.5'
 '5914.17' '2620.02' '833.96' '819.34' '1471.38' '850.11' '1306.49'
 '754.23' '21474.95' '13201.41' '6387.21' '18083.44' '873.58' '17537.98'
 '6000.77' '10833.19' '16020.03' '7114.26' '6030.42' '2454.9' '16114.49'
 '11391.17' '13605.82' '1614.06' '6057.22' '24768.97' '57355.37'
 '10961.98' 'unknown' '1567.56' '2591.0' '2649.6' '19413.45' '10473.59'
 '19526.23' '18796.73' '12488.11' '6374.37' '20060.81' '1497.44' '747.12'
 '19188.4' '0.01' '768.56' '1396.91' '1609.3' '914.39' '766.61' '913.83'
 '18399.95' '62277.55' '6650.19' '15939.64' '13470.62' '15314.45'
 '2655.22' '838.68' '24155.77' '709.84' '5681.86' '50527.25' '54513.26'
 '6441.52' '21416.86' '6138.31' '5630.7' '5921.94' '20554.99' '53481.85'
 '6517.3' '5807.05' '19472.45' '58729.84' '6469.47' '7220.72' '11345.43'
 '5771.18' '20216.39' '19520.24' '19902.64' '47878.94' '12937.33'
 '52102.18' '22205.7' '12659.28' '741.28' '20404.27' '5913.47' '1408.14'
 '815.82' '825.7' '24662.46' '52770.21' '19572.25' '24732.72' '20499.62'
 '1630.81' '1538.05' '6733.12' '6791.02' '5714.52' '12872.17' '10317.78'
 '1716.42' '10883.31' '59732.11' '16418.65' '60913.7' '16011.74'
 '19045.83' '7042.13' '51726.9' '11964.97' '16785.09' '12110.0' '19192.54'
 '13178.71' '12594.39' '19791.57' '19095.87' '13344.99' '1535.06'
 '19865.02' '13708.37' '723.3' '22437.72' '16461.72' '866.44' '23679.28']

Unique values in 'CustomerAge':
['61.0' '50.0' '35.0' '66.0' '34.0' '53.0' '23.0' '28.0' '56.0' '32.0'
```

```
 '27.0' '55.0' '25.0' '47.0' '67.0' '64.0' '58.0' '18.0' '33.0' '45.0'
 '46.0' '31.0' '24.0' '21.0' '26.0' '69.0' '42.0' '60.0' '65.0' '51.0'
 '43.0' '54.0' '40.0' '63.0' nan '52.0' '68.0' '185.0' '37.0' '39.0'
 'forty' '57.0' '48.0' '20.0' '30.0' '19.0' '29.0' '70.0' '38.0' '36.0'
 '62.0' '22.0' '29 years' '3.0' '59.0']

Unique values in 'City':
['Mumbai' ' delhi ' 'Delhi' 'Chennai' 'Bengaluru' 'MUMBAI' 'Pune'
 'Kolkata' 'pUnE' 'Gurugram' 'Noida' '  bengaluru' nan]

Unique values in 'OrderStatus':
['Completed' 'Pending' 'Delivered' 'Shipped' 'Cancelled' nan 'completed '
 'SHIPPED' ' pending']
```

```
cleaned_df = df.copy()
print("Copy created successfully!")
```

```
Copy created successfully!
```

```
for col in cleaned_df.select_dtypes(include='object').columns:
    cleaned_df[col] = cleaned_df[col].astype(str).str.strip()

cleaned_df['City'] = cleaned_df['City'].str.title()
cleaned_df['OrderStatus'] = cleaned_df['OrderStatus'].str.title()

print("Text cleaning completed!")
```

```
Text cleaning completed!
```

```
# Correct Quantity
cleaned_df['Quantity'] = (
    cleaned_df['Quantity']
    .replace({'three': '3', '2 pcs': '2'})
)

cleaned_df['Quantity'] = pd.to_numeric(
    cleaned_df['Quantity'], errors='coerce'
)

# Correct UnitPrice
cleaned_df['UnitPrice'] = (
    cleaned_df['UnitPrice']
    .replace('unknown', None)
    .str.replace(',', '', regex=False)
)

cleaned_df['UnitPrice'] = pd.to_numeric(
    cleaned_df['UnitPrice'], errors='coerce'
)

# Correct CustomerAge
cleaned_df['CustomerAge'] = (
    cleaned_df['CustomerAge']
    .replace('forty', '40')
)

cleaned_df['CustomerAge'] = pd.to_numeric(
    cleaned_df['CustomerAge'], errors='coerce'
)
```

```
print("Numerical errors corrected!")
```

```
Numerical errors corrected!
```

```
cleaned_df['CustomerAge'] = (
    df['CustomerAge']
    .astype('string')
    .str.strip()
    .str.lower()
    .replace({'forty': '40'})
    .str.replace(r'\s*years?$', '', regex=True)
)

cleaned_df['CustomerAge'] = pd.to_numeric(
    cleaned_df['CustomerAge'], errors='coerce'
)

print("Customer ages corrected!")
```

```
Customer ages corrected!
```

```
before = len(cleaned_df)

cleaned_df = cleaned_df.drop_duplicates()

after = len(cleaned_df)

print("Duplicates removed:", before - after)
print("Rows remaining:", after)
```

```
Duplicates removed: 5
Rows remaining: 180
```

```
# Print the data types of the cleaned numerical columns
print("Data Types:")
print(cleaned_df[['Quantity', 'UnitPrice', 'CustomerAge']].dtypes)

# Print the summary of missing values in the cleaned dataset
print("\nMissing values:")
print(cleaned_df.isnull().sum())
```

```
Data Types:
Quantity       float64
UnitPrice      float64
CustomerAge    Float64
dtype: object

Missing values:
OrderID         0
CustomerName    0
City            0
Product         0
Quantity        2
UnitPrice       3
OrderDate       0
OrderStatus     0
CustomerAge     2
dtype: int64
```

```
import pandas as pd
import matplotlib.pyplot as plt

# 1. Handle missing categorical values
for col in ['CustomerName', 'City', 'Product', 'OrderStatus']:
    cleaned_df[col] = cleaned_df[col].fillna('Unknown')

# 2. Standardize text
cleaned_df['City'] = cleaned_df['City'].astype(str).str.strip().str.title()
cleaned_df['OrderStatus'] = cleaned_df['OrderStatus'].astype(str).str.strip().str.title

# 3. Convert dates
cleaned_df['OrderDate'] = pd.to_datetime(
    cleaned_df['OrderDate'], errors='coerce', dayfirst=True
)

# 4. Handle unusual values (outliers)
cleaned_df.loc[
    (cleaned_df['Quantity'] < 0) | (cleaned_df['Quantity'] > 100), 'Quantity'
] = float('nan')

cleaned_df.loc[
    (cleaned_df['UnitPrice'] < 0) | (cleaned_df['UnitPrice'] > 1000000), 'UnitPrice'
] = float('nan')

cleaned_df.loc[
    (cleaned_df['CustomerAge'] < 0) | (cleaned_df['CustomerAge'] > 120), 'CustomerAge'
] = float('nan')

# 5. Calculate total amount
cleaned_df['TotalAmount'] = (
    cleaned_df['Quantity'] * cleaned_df['UnitPrice']
)

# 6. Display final dataset information
print("PROJECT CLEANING COMPLETED!")
print("--------------------------------")
print("Original rows:", len(df))
print("Final rows:", len(cleaned_df))
print("Columns:", len(cleaned_df.columns))

print("\nRemaining missing values:")
print(cleaned_df.isnull().sum())

print("\nFinal dataset:")
display(cleaned_df.head())

print("\nSummary statistics:")
display(cleaned_df.describe())

# 7. Create charts
cleaned_df['OrderStatus'].value_counts().plot(
    kind='bar', title='Orders by Status'
)
plt.xlabel('Order Status')
plt.ylabel('Number of Orders')
plt.show()

cleaned_df['City'].value_counts().plot(
    kind='bar', title='Orders by City'
```

```
)
plt.xlabel('City')
plt.ylabel('Number of Orders')
plt.show()

# 8. Save cleaned dataset
cleaned_df.to_csv('cleaned_sales_dataset.csv', index=False)
print("\nYour cleaned CSV file has been saved as 'cleaned_sales_dataset.csv'!")
```

```
PROJECT CLEANING COMPLETED!
```

```
from google.colab import files

files.download('cleaned_sales_dataset.csv')
```

```
Remaining missing values:
OrderID         0
```

```
# 1. Replace empty or nan-like string representations in the City column with 'Unknown'
cleaned_df['City'] = cleaned_df['City'].replace(
    ['nan', 'Nan', 'NaN', ''], 'Unknown'
)

# 2. Save the final cleaned dataset
cleaned_df.to_csv('cleaned_sales_dataset.csv', index=False)
print("Saved cleaned dataset successfully with resolved 'City' values!")
```

```
Saved cleaned dataset successfully with resolved 'City' values!
Final dataset:
/tmp/ipykernel_1485/2775876478.py:13: UserWarning: Parsing dates in %Y-%m-%d format when
```

```
from google.colab import files
files.download('cleaned_sales_dataset.csv')
```

0	10001	Nikhil Jain	Mumbai	Laptop	3.0	30790.72	2026-01-18	Completed
1	10002	Simran Kaur	Delhi	Monitor	1.0	10307.27	2026-01-28	Pending
2	10003	Zoya Ali	Delhi	Headphones	5.0	2439.64	2026-02-27	Delivered
3	10004	Priya Bansal	Delhi	Tablet	4.0	20945.65	NaT	Pending
4	10005	nan	Mumbai	Phone	4.0	15822.27	2026-02-14	Delivered

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

```
Summary statistics:
```

	OrderID	Quantity	UnitPrice	OrderDate	CustomerAge	TotalAmount
count	180.000000	176.000000	177.000000	175	177.0	1.730000e+02
mean	10090.500000	2.903409	19824.887062	2026-02-18 17:16:48	44.163842	6.534472e+04
min	10001.000000	1.000000	0.010000	2026-01-01 00:00:00	3.0	4.000000e-02
25%	10045.750000	2.000000	2454.900000	2026-01-28 00:00:00	31.0	5.714520e+03
50%	10090.500000	3.000000	10883.310000	2026-02-21 00:00:00	45.0	2.466246e+04
75%	10135.250000	4.000000	19472.450000	2026-03-12 12:00:00	57.0	5.970792e+04
max	10180.000000	5.000000	999999.000000	2026-03-31 00:00:00	70.0	3.999996e+06
std	52.105662	1.429002	75649.619694	NaN	15.442019	3.058904e+05